

INSLIB — Getting Started in Python

The navigation filter in a few lines

Jan Zwiener

2026

Contents

1	INSLIB in Python	1
1.1	Install	1
1.2	Your first script	2
1.3	The per-epoch loop, in three moves	2
1.4	Reading the Solution	3
1.5	Starting without GNSS: Lighthouse / UWB / mocap	4
1.6	Referencing yaw to true north (magnetometer)	4
1.7	The lean alternative: Ins	4
1.8	Watching it live	5
1.9	Where to look next	5

1 INSLIB in Python

The same navigation filter you get in C is available as a small Python package, **INSLIB**. It is a thin **ctypes** binding over the compiled library plus some conveniences (telemetry to PlotJuggler / MAVLink, a YAML CSV runner). This tutorial shows how little code it takes to get a solution.

If you have not read it yet, the concepts (frames, states, the “feed measurements, read a solution” loop) are introduced in the companion `c_tutorial.md`. Here we focus on the Python surface.

1.1 Install

Today the package installs as an **editable** install against a prebuilt shared library:

```
make pylib          # builds python/INSLIB/libINSLIB.so
pip install -e python/  # now `import INSLIB` works anywhere
```

Is pip install INSLIB coming? Yes, that is the intended end state. The packaging is already in place (`python/pyproject.toml`), and because the binding is **ctypes** over a flat C ABI, a released wheel is *one wheel per platform*, not per Python version. The remaining piece is a build backend that compiles the C sources into the wheel. Until then, the `make pylib` + editable-install path above is the supported way.

You need Python 3.8 or newer and a C compiler for `make pylib`. Everything in this tutorial uses only the standard library plus **INSLIB**.

1.2 Your first script

Navigator is the high-level entry point: push whatever sensors you have each epoch, then read a unified Solution. Here is the Python twin of the C tutorial's first program: a stationary IMU at 100 Hz with a once-per-second GNSS fix:

```
import math
from INSLIB import Navigator, Config, llh_to_ecef, ecef_to_llh

D2R = math.pi / 180.0

# The (stationary) location: a spot in Munich, 520 m ellipsoidal.
lat, lon, h = 48.1372 * D2R, 11.5756 * D2R, 520.0
ecef = llh_to_ecef(lat, lon, h)

nav = Navigator(Config(auto_init=True))

dt = 0.01 # 100 Hz
for k in range(1, 601):
    t_us = int(k * dt * 1e6)
    nav.imu(t_us, dt, acc=(0.0, 0.0, -9.81), gyr=(0.0, 0.0, 0.0))
    if k % 100 == 0: # a 1 Hz GNSS fix
        nav.gnss_pos(ecef, var_ned=(4.0, 4.0, 16.0)) # 2/2/4 m stddev
    nav.update() # run the filter for this epoch
    sol = nav.solution() # read the result back

print("mode:", sol.mode, " ready:", sol.ready)
out = ecef_to_llh(*sol.pos_ecef)
print("lat %.6f deg lon %.6f deg h %.1f m"
      % (out[0] / D2R, out[1] / D2R, out[2]))
print("rpy %.2f %.2f %.2f deg"
      % (sol.roll / D2R, sol.pitch / D2R, sol.yaw / D2R))
```

Run it:

```
python3 hello.py
```

Expected output:

```
mode: FULL ready: True
lat 48.137200 deg lon 11.575600 deg h 520.0 m
rpy -0.00 -0.00 0.00 deg
```

1.3 The per-epoch loop, in three moves

```
nav.imu(t_us, dt, acc, gyr) # 1. begin the epoch with the IMU sample
nav.gnss_pos(ecef, var_ned) # add whatever aiding you have this epoch
nav.update() # 2. run the filter
sol = nav.solution() # 3. read the result
```

The one gotcha: `nav.solution()` (and `nav.state()`, `nav.rpy()`, ...) are *readers*. They do not run the filter. You must call `nav.update()` **first** each epoch, after pushing the sensors. Forget it and the solution never leaves `mode=NONE`.

Add only the sensors you have, each is a one-liner:

```
nav.imu(t_us, dt, acc, gyr)           # begins an epoch (var args optional)
nav.gnss_pos(ecef, var_ned)           # ECEF fix + NED covariance
nav.gnss_pos(ecef, var_ned, delay_ms=80) # ...or a late fix (80 ms here, history-anchored)
nav.gnss_vel(vel_ned, var_ned)       # NED velocity
nav.mag(mag_uT, mag_var)             # magnetometer
nav.baro(pressure_pa)                # static pressure -> vertical channel
nav.yaw(yaw_rad, stddev_rad)         # absolute heading (e.g. dual-antenna, pose)
nav.local_pos(pos_ned, var_ned)      # Lighthouse / UWB / mocap
nav.zupt(True); nav.zaru(True)       # known standstill / clamp
```

`var_ned` is either a 3-element NED variance diagonal (as above) or a full 3×3 covariance (nested rows or flat row-major length 9).

`nav.yaw()` takes **radians** in either convention, `[-pi, pi]` or `[0, 2*pi]`. A heading beyond one full turn is treated as a unit or unwrapping mistake, dropped, and counted in the filter's `n_invalid_input` diagnostic instead of being wrapped into a wrong heading.

1.4 Reading the Solution

`nav.solution()` returns a `Solution` dataclass:

```
sol.mode           # "FULL" / "COASTING" / "ATTITUDE_ONLY" / "NONE"
sol.ready          # position/velocity usable?
sol.roll, sol.pitch, sol.yaw # best-available attitude [rad]
sol.pos_ecef, sol.pos_local  # WGS84 ECEF [m], local NED [m]
sol.vel_ned        # NED velocity [m/s]
sol.lat_rad, sol.lon_rad, sol.alt_m
sol.height_m       # best height above the datum (ins, else baro)
sol.height_ell_m   # best absolute (ellipsoid) height
```

The mode tells you how good the answer is right now. Navigator degrades gracefully: with fresh GNSS you get `FULL`, through an outage it `COASTINGs` on the IMU, if position is lost entirely, `ATTITUDE_ONLY` still gives you roll/pitch/yaw from the attitude-only (AHRS/ARS) fallback.

Reaching `FULL` in the first place takes fixes that are better than merely fusable: the entry gate (`Config.gnss_start_max_*`) has to hold for `gnss_init_dwell_sec`. Raise those if your receiver reports a coarser accuracy, otherwise the filter never leaves `ATTITUDE_ONLY` (the log says so once per 30 s). The reverse holds too: if every fix for `gnss_stop_dwell_sec` is worse than the exit gate (`gnss_stop_max_*`), the mode drops back to `ATTITUDE_ONLY` and the filter stops - `pos_local` and `pos_ecef` go `None`, so nothing keeps drifting off bad fixes. Only a full re-bootstrap through the entry gate brings `FULL` back, carrying the IMU biases over at an inflated uncertainty and re-seeding attitude from the AHRS. Set `gnss_stop_disable=True` to switch the exit off entirely, or `auto_reacquire_disable=True` to keep the filter running and merely have `ready` go false.

1.5 Starting without GNSS: Lighthouse / UWB / mocap

No GNSS? Aid the filter with a local NED position instead. Give the lab's coarse location once in the `Config` (for gravity and true north only, not as an aiding measurement), then push `local_pos` each epoch:

```
import math
from INSLIB import Navigator, Config

D2R = math.pi / 180.0

# Coarse lab anchor - gravity + north reference, not an aiding fix.
nav = Navigator(Config(auto_init=True,
                      lat_rad=48.1372 * D2R, lon_rad=11.5756 * D2R,
                      h_m=520.0))

dt = 0.01
for k in range(1, 601):
    nav.imu(int(k * dt * 1e6), dt, acc=(0.0, 0.0, -9.81), gyr=(0.0, 0.0, 0.0))
    if k % 10 == 0:
        # a 10 Hz indoor tracker
        nav.local_pos((1.5, 0.0, 0.0), var_ned=(1e-4, 1e-4, 1e-4)) # 1 cm
    nav.update()
    sol = nav.solution()

print(sol.mode, sol.ready, tuple(round(x, 3) for x in sol.pos_local))
# -> FULL True (1.5, -0.0, -0.0)
```

The position you pass must already be in the filter's NED frame (origin = the `Config` lat/lon/h). Aligning your tracker's frame to it is up to you.

1.6 Referencing yaw to true north (magnetometer)

If you have a magnetometer, arm the World Magnetic Model once (e.g. on the first coarse position) so estimated yaw is relative to true north and the magnetic-disturbance gate is active:

```
nav.set_magnetic_model(lat_rad, lon_rad, 2026.5) # decimal year
...
nav.mag(mag_uT, (0.0, 0.0, 0.0)) # per-axis variance, 0 -> library default
```

Pass zeros for the variance unless you have characterised the sensor *on the target platform*. The library's own default is deliberately weak and the fusion is throttled to 1 Hz (`Config(magnetometer_min_delay_ms=...)`, negative switches the throttle off). The magnetometer is there to keep yaw from drifting away over minutes, over seconds the gyro is the better instrument.

1.7 The lean alternative: `Ins`

If you do not need the AHRS/baro fallback, `Ins` is the bare 15-state ESKF the same three-move loop, a little leaner. Note it uses `update()` and then plain accessors rather than a `Solution`:

```

from INSLIB import Ins, Config, llh_to_ecef

nav = Ins(Config(auto_init=True))
nav.imu(t_us, dt, acc, gyr)
nav.gnss_pos(llh_to_ecef(lat, lon, h), var_ned=(4.0, 4.0, 16.0))
nav.update()
print(nav.rpy(), nav.is_ready(), nav.position_local())

```

Conventions match the C library exactly: body frame FRD, nav frame NED, Hamilton quaternion $q = [w, x, y, z]$, time in int64 microseconds, angles in radians.

1.8 Watching it live

INSLIB can stream the estimate straight into [PlotJuggler](#) (UDP/JSON) and to any MAVLink GCS:

```

from INSLIB import Telemetry
tele = Telemetry(plotjuggler=True, mavlink=False)
...
nav.update(); tele.publish(nav.state())

```

For a full end-to-end demo against a real dataset (with a suggested PlotJuggler layout), see `python/DEMO.md`. To run your own CSV logs without writing any code at all, there is a YAML-configured runner:

```
python3 -m INSLIB myrun.yaml # or: inslib-run myrun.yaml
```

See `python/README.md` and `python/examples/runner.yaml` for that path.

1.9 Where to look next

- `python/README.md` - the full binding reference (telemetry, runner, Allan-variance tool, post-processing GUI).
- `python/DEMO.md` - live replay + PlotJuggler walkthrough.
- `c_tutorial.md` - the underlying library and its conventions.